

CAR RENTAL MONOLITH BACKEND

Giải thích toàn bộ file server back-end

Spring Boot modular monolith, database migration, luồng API vehicle và đánh giá SOLID

Workspace

C:/source/car-rental-monolith/server

Ngày tạo

22/05/2026

Phạm vi

Source, config, infra, docs, migration, test

1. Phạm vi tài liệu

Tài liệu này giải thích phần server back-end nằm trong thư mục C:/source/car-rental-monolith/server. Nội dung tập trung vào source code, cấu hình runtime, migration database, infra local, tài liệu kiến trúc và test. Thư mục target là output Maven sinh ra nên không phân tích như source chính. Các file IDE/tool local vẫn được liệt kê riêng để rõ vì sao chúng tồn tại nhưng chúng không tham gia runtime backend.

Tóm tắt nhanh

Backend là Spring Boot monolith. Source được tổ chức theo hướng modular monolith: một app deploy duy nhất, nhưng code và database chia theo module nghiệp vụ như vehicle, booking, payment, customer. Module Java hoàn chỉnh nhất hiện tại là vehicle. Các module khác đã có schema Flyway nhưng phần Java còn là khung thư mục.

Cách đọc tài liệu

- Đọc mục tổng quan kiến trúc trước để hiểu luồng request và ranh giới module.
- Đọc bảng file-by-file để biết từng file làm gì và tại sao cần tồn tại.
- Đọc mục SOLID để hiểu code hiện tại áp dụng nguyên tắc thiết kế ra sao và chỗ nào cần cải thiện.

2. Tổng quan kiến trúc backend

Project dùng Spring Boot 4.0.6, Maven, Java 26, Spring MVC, Spring Data JPA, Flyway, PostgreSQL, P6Spy, Actuator và Prometheus registry. Cách chia source hiện tại thể hiện định hướng hexagonal/ports-and-adapters bên trong modular monolith.

```
HTTP request từ admin web
-> AdminVehicleController
-> ListVehiclesUseCase / PageVehiclesUseCase
-> VehicleQueryService
-> LoadVehiclePort
-> VehiclePersistenceAdapter
-> VehicleJpaRepository
-> PostgreSQL: vehicle.vehicles
-> VehicleJpaEntity
-> Vehicle domain
-> VehicleResponse
-> ApiResponse JSON
```

Lý do không để controller gọi thẳng repository: controller chỉ nên biết HTTP; service/use case điều phối nghiệp vụ; repository/JPA là chi tiết hạ tầng; port/interface giúp thay đổi persistence hoặc test mock dễ hơn. Đây là điểm quan trọng nhất để hiểu SOLID trong code hiện tại.

Module và schema

Database dùng schema-per-module. Ví dụ bảng xe nằm trong schema vehicle, booking nằm trong schema booking, thanh toán nằm trong schema payment. Cross-module reference chủ yếu dùng UUID và snapshot thay vì foreign key vật lý xuyên schema nghiệp vụ. Cách này giảm coupling database và chuẩn bị cho khả năng tách service trong tương lai.

3. Build, dependency và runtime

pom.xml

pom.xml là file build quan trọng nhất. Nó khai báo Spring Boot parent 4.0.6, Java 26, artifact car-rental-monolith và toàn bộ dependency backend. Các dependency chính gồm webmvc cho REST API, data-jpa cho repository, validation cho request validation, actuator và prometheus cho vận hành, Flyway cho database migration, postgresql driver để kết nối DB, p6spy để log SQL, lombok để giảm boilerplate entity, devtools cho local development.

Dependency	Vai trò	Vì sao có trong backend
spring-boot-starter-webmvc	REST API bằng Spring MVC	AdminVehicleController cần nhận HTTP request và trả JSON.
spring-boot-starter-data-jpa	JPA/Hibernate + Spring Data repository	VehicleJpaRepository đọc bảng vehicle.vehicles.
spring-boot-starter-validation	Bean validation	RestExceptionHandler đã chuẩn bị xử lý MethodArgumentNotValidException.
spring-boot-starter-actuator	Health, metrics, operational endpoints	Cần cho monitoring khi app chạy thật.
micrometer-registry-prometheus	Export metrics Prometheus	Kết hợp Actuator để giám sát.
flyway-core + flyway-database-postgresql	Versioned database migration	Schema do SQL migration quản lý, không để Hibernate tự sửa DB.
postgresql	JDBC driver PostgreSQL	Datasource cần driver để kết nối DB.
p6spy	SQL logging proxy	Debug SQL thực tế Hibernate gửi xuống DB.
lombok	Sinh getter/setter/constructor khi compile	VehicleJpaEntity dùng Lombok để giảm boilerplate.
aspectjweaver	AOP weaving support	RepositoryLoggingAspect dùng @Aspect để gắn context repository vào SQL log.

application.yaml

application.yaml tách cấu hình runtime khỏi Java code. Datasource mặc định trỏ tới jdbc:p6spy:postgresql://localhost:5432/car_rental với user/password local. HikariCP được cấu hình pool 20 connection, minimum idle 5, auto-commit false và leak detection 60s. Flyway bật ở classpath:db/migration. Hibernate ddl-auto none nghĩa là không tự tạo/sửa schema; mọi thay đổi DB phải qua Flyway migration. open-in-view false tránh giữ Hibernate session xuyên suốt request, buộc service layer kiểm soát transaction rõ hơn.

spy.properties và SQL logging

spy.properties chọn P6SpySqlFormatter làm formatter. RepositoryLoggingAspect dùng AOP bắt các bean tên kết thúc bằng Repository, lưu tên repository vào RepositoryQueryContext ThreadLocal, sau đó P6SpySqlFormatter in SQL kèm tên repository. Đây là ví dụ tách cross-cutting concern logging khỏi code repository.

4. Giải thích file gốc và tooling

File	Vai trò	Tại sao cần	SOLID / ghi chú
.gitattributes	Cấu hình line ending cho script Maven wrapper: mvnw dùng LF, file .cmd dùng CRLF.	Giúp script chạy ổn định trên Linux/macOS và Windows. Không ảnh hưởng logic runtime nhưng quan trọng cho build đa môi trường.	SRP ở mức repo: chỉ xử lý quy ước text/binary, không trộn với cấu hình build.
.gitignore	Loại trừ target, build output, IDE metadata, Maven wrapper jar và cache tooling.	Giữ repository sạch, tránh commit file sinh ra từ máy cá nhân hoặc từ quá trình build.	Không trực tiếp liên quan SOLID; đây là vệ sinh source control.
.mvn/wrapper/maven-wrapper.properties	Khai báo Maven Wrapper version 3.3.4 và Maven distribution 3.9.15.	Cho phép chạy ./mvnw hoặc mvnw.cmd mà không yêu cầu mọi máy cài sẵn Maven đúng version.	Tách dependency build khỏi môi trường cá nhân.
mvnw	Script Maven Wrapper cho Linux/macOS.	Chuẩn hóa lệnh build/test trên môi trường Unix-like và CI.	Không chứa business logic.
mvnw.cmd	Script Maven Wrapper cho Windows.	Phù hợp workspace hiện tại trên Windows và giúp dev Windows build cùng cấu hình.	Không chứa business logic.
pom.xml	File Maven chính: Spring Boot parent 4.0.6, Java 26, dependency webmvc/data-jpa/validation/actuator/Flyway/PoStgreSQL/P6Spy/Lombok và plugin build.	Đây là trung tâm khai báo cách build, chạy và đóng gói backend.	Hỗ trợ DIP/OCF gián tiếp bằng cách đưa framework/infrastructure vào project nhưng code vẫn nên phụ thuộc interface ở application layer.
README.md	README hiện chỉ có tên project car-rental-monolith.	Là nơi nên đặt hướng dẫn chạy nhanh, hiện còn rất tối giản.	Không liên quan trực tiếp SOLID.
HELP.md	Tài liệu do Spring Initializr sinh ra, gồm các link tham khảo Maven, Spring Data JPA, Spring Web, Actuator, Prometheus.	Hữu ích lúc khởi tạo project; không phải tài liệu kiến trúc chính của team.	Không liên quan trực tiếp SOLID.

File IDE/tool local

Nhóm file dưới đây có trong server nhưng không phải backend runtime. Chúng giải thích cách IDE/tool local mở project. Khi đánh giá source production, không nên để logic dự án phụ thuộc các file này.

File	Vai trò	Tại sao cần	SOLID / ghi chú
.claude/settings.local.json	Cấu hình local cho công cụ hỗ trợ phát triển.	Phục vụ môi trường cá nhân, không tham gia build/runtime backend.	Nên tránh phụ thuộc logic dự án vào file local này.
.idea/.gitignore	Cấu hình ignore riêng trong thư mục IntelliJ IDEA.	Giúp IDE không đưa các file local nhạy cảm hoặc tạm vào VCS.	Không liên quan runtime.
.idea/compiler.xml	Cấu hình compiler trong IntelliJ IDEA.	Giúp IDE biết cách compile project khi mở bằng IntelliJ.	Không thay thế pom.xml; build chính vẫn phải do Maven quyết định.
.idea/encodings.xml	Cấu hình encoding trong IDE.	Tránh lỗi tiếng Việt/ký tự đặc biệt khi mở file trong IntelliJ.	Không liên quan runtime.
.idea/jarRepositories.xml	Cấu hình repository jar mà IntelliJ biết.	Giúp IDE resolve dependency Maven.	Không thay thế cấu hình dependency trong pom.xml.

File	Vai trò	Tại sao cần	SOLID / ghi chú
.idea/misc.xml	Cấu hình project-level của IntelliJ.	Lưu JDK/project metadata cho IDE.	Không liên quan runtime.
.idea/modules.xml	Danh sách module IntelliJ.	Giúp IDE map project/module.	Không liên quan runtime.
.idea/vcs.xml	Cấu hình VCS trong IDE.	Giúp IntelliJ hiểu repository dùng Git.	Không liên quan runtime.
.idea/workspace.xml	Trạng thái workspace cá nhân của IDE.	Thường là file local, không nên coi là tài liệu backend.	Không liên quan runtime.

5. Infra local

File	Vai trò	Tại sao cần	SOLID / ghi chú
infra/docker/docker-compose.yml	Định nghĩa môi trường local gồm postgres:16-alpine và pgAdmin 4, volume dữ liệu và bridge network.	Giúp dev chạy database đúng với application.yml mà không phải cài PostgreSQL thủ công. Healthcheck đảm bảo pgAdmin chờ Postgres sẵn sàng.	Tách hạ tầng local khỏi code Java. Đây là separation of concerns ở mức vận hành.

docker-compose.yml tạo Postgres và pgAdmin. Đây là cách tốt cho local development vì database khớp với application.yml và có volume để giữ dữ liệu giữa các lần restart.

6. Tài nguyên runtime và database migration

Resources

File / thư mục	Vai trò	Tại sao cần	SOLID / ghi chú
src/main/resources/application.yml	Cấu hình runtime: tên app, datasource PostgreSQL qua P6Spy, HikariCP, Flyway, JPA/Hibernate và logging.	Tách cấu hình khỏi code Java. Dùng biến môi trường DB_URL/DB_USERNAME/DB_PASSWORD để dev/prod thay đổi mà không sửa source.	Hỗ trợ OCP ở mức cấu hình: đổi DB URL/driver/pool/logging mà không sửa class business.
src/main/resources/spy.properties	Cấu hình P6Spy: logger SLF4J, formatter custom P6SpySqlFormatter, loại bớt category log nhiễu.	Cho log SQL dễ đọc khi debug JPA/Hibernate, nhưng tránh quá nhiều log result/resultset.	Kết hợp với P6SpySqlFormatter theo SRP: properties chọn formatter, formatter xử lý format.
src/main/resources/static/	Thư mục convention của Spring Boot cho static assets, hiện rỗng.	Nếu backend cần phục vụ file tĩnh trực tiếp thì đặt ở đây; hiện admin web nằm riêng nên chưa dùng.	Không liên quan SOLID hiện tại.
src/main/resources/templates/	Thư mục convention cho server-rendered templates, hiện rỗng.	Dành cho Thymeleaf/MVC template nếu sau này backend render HTML. Hiện project là REST API nên chưa dùng.	Không liên quan SOLID hiện tại.

Flyway migrations và seed

Flyway chạy file theo thứ tự version V001, V005, V010... V110. Mỗi migration đã chạy được ghi vào flyway_schema_history, lần sau app start chỉ chạy migration mới. Vì ddl-auto là none, đây là nguồn sự thật của schema database.

File	Tạo / làm gì	Tại sao cần	SOLID / ghi chú
V001__create_extensions_and_schemas.sql	Bật pgcrypto và tạo schema common, identity, customer, vehicle, fleet, driver, pricing, booking, payment, review, notification, admin.	Đặt nền móng schema-per-module. pgcrypto cung cấp gen_random_uuid() để DB tự sinh UUID.	SRP kiến trúc: mỗi schema là boundary nghiệp vụ.
V005__common.sql	Tạo common.outbox_events và common.idempotency_keys.	Outbox lưu event sau transaction để publish an toàn; idempotency chống xử lý trùng request khi client retry.	Tách cross-cutting infrastructure khỏi module nghiệp vụ.
V010__identity.sql	Tạo users, roles, permissions, user_roles, role_permissions, login_sessions cùng unique/index liên quan.	Quản lý tài khoản, phân quyền và session độc lập với customer/admin/driver.	SRP: identity chỉ lo định danh và quyền.
V020__customer.sql	Tạo customers, customer_roles, host_profiles, kyc_profiles, kyc_documents, addresses.	Quản lý hồ sơ khách hàng, vai trò renter/host, KYC và địa chỉ.	Tách domain customer khỏi identity và vehicle.
V030__vehicle.sql	Tạo vehicles, vehicle_listings, vehicle_images, vehicle_features, availability_blocks cùng index.	Lưu thông tin xe, tin cho thuê, ảnh, tiện ích và khoảng khóa lịch.	Đây là schema được module Java vehicle hiện tại sử dụng trực tiếp.
V031__vehicle_search_indexes.sql	Bật pg_trgm và tạo GIN trigram indexes cho brand, model, version, license_plate.	Tối ưu search ILIKE %keyword% trong VehicleJpaRepository.search.	OCP hiệu năng: tối ưu DB mà không cần đổi API contract.
V040__fleet.sql	Tạo branches, company_vehicles, maintenance_records, insurance_policies, inspection_records.	Quản lý xe công ty, chi nhánh, bảo trì, bảo hiểm và kiểm định.	Tách fleet asset khỏi listing public ở vehicle.
V050__driver.sql	Tạo drivers, driver_documents, availability_slots, driver_assignments.	Hỗ trợ dịch vụ thuê xe có tài xế: hồ sơ tài xế, giấy tờ, lịch rảnh và phân công booking.	Module driver có schema riêng, chưa có Java implementation.
V060__pricing.sql	Tạo price_plans, promotions, quotes, quote_items, promotion_redemptions.	Tách tính giá/báo giá/khuyến mãi khỏi booking để có quote trước khi đặt xe.	SRP: pricing tính tiền, booking quản lý vòng đời đơn.
V070__booking.sql	Tạo bookings, booking_status_history, trip_checklists, trip_checklist_items, booking_cancellations.	Booking là trung tâm vận hành, lưu snapshot xe/khách/tài xế để booking cũ ổn định dù profile thay đổi.	Giảm coupling cross-module bằng UUID reference và snapshot.
V080__payment.sql	Tạo payment_intents, transactions, refunds, payouts.	Quản lý ý định thanh toán, giao dịch thực tế, hoàn tiền và chi trả.	SRP: payment không xử lý lifecycle booking.
V090__review.sql	Tạo reviews, review_replies, review_reports.	Lưu đánh giá sau chuyến đi, phản hồi và báo cáo review.	Tách review khỏi booking để giảm phụ thuộc.
V100__notification.sql	Tạo templates, notifications, delivery_logs, devices.	Quản lý mẫu thông báo, bản ghi gửi, log provider và device token.	Tách notification/integration khỏi module nghiệp vụ.
V110__admin.sql	Tạo audit_logs, backoffice_tasks, dashboard_snapshots.	Hỗ trợ backoffice: truy vết hành động, task vận hành và snapshot dashboard.	Tách admin/operation khỏi core booking.
src/main/resources/db/seed/dev/seed_vehicles.sql	Seed 100 xe mẫu từ template, có HOST_OWNED/COMPANY_OWNED, nhiều status và ON CONFLICT license_plate DO NOTHING.	Dữ liệu dev/test để admin web có danh sách xe kiểm thử. Không đặt trong migration chính để tránh tự động nạp data mẫu vào mọi môi trường.	Tách data dev khỏi schema production.

7. Java shared và entry point

Package gốc là com.ares.car_rental_monolith. Dấu gạch ngang không hợp lệ trong Java package, vì vậy tên project car-rental-monolith được chuyển thành package có underscore.

File	Vai trò	Tại sao cần	SOLID / ghi chú
CarRentalMonolithApplication.java	Entry point Spring Boot với @SpringBootApplication và SpringApplication.run.	Khởi động Spring context, auto-configuration, component scan, controller/service/repository.	SRP: chỉ boot app, không chứa business logic.
shared/api/ApiResponse.java	Record envelope chuẩn cho response: success, code, message, data, timestamp; có factory success/error.	Giúp frontend nhận format API nhất quán cho cả thành công và lỗi.	SRP: chỉ định nghĩa response envelope.
shared/api/ListResponse.java	Record cho list không phân trang: items và total, factory of tự tính total.	API list trả về tổng số item nhất quán, frontend không tự suy luận.	SRP: response list đơn giản.
shared/api/PageResponse.java	Record cho phân trang: items, total, page, size, totalPages, hasNext, hasPrevious; có map() đổi kiểu item.	Giữ metadata phân trang khi controller map domain sang DTO.	OCP nhỏ: map() cho phép đổi representation mà không viết lại pagination metadata.
shared/config/CorsConfig.java	Cấu hình CORS cho /api/**, origin localhost/127.0.0.1:5173, các HTTP method phổ biến.	Cho admin web Vite local gọi backend khác origin trong browser.	SRP: chỉ lo CORS, không lẫn security/business.
shared/config/P6SpySqlFormatter.java	Formatter custom cho SQL log: bỏ SQL rỗng, in elapsed time/category/repository caller, format select/insert/update/delete.	Log SQL từ Hibernate/P6Spy để đọc hơn khi debug query.	SRP tương đối tốt; nếu format phức tạp hơn có thể tách strategy từng loại SQL.
shared/config/RepositoryLoggingAspect.java	Aspect intercept bean *Repository, set tên repository vào RepositoryQueryContext trước khi query chạy.	Giúp P6SpySqlFormatter biết SQL được gọi từ repository nào.	AOP tách logging concern khỏi repository. Đây là separation of concerns tốt.
shared/config/RepositoryQueryContext.java	ThreadLocal giữ tên repository hiện tại, có set/get/clear.	Truyền context từ aspect sang SQL formatter trong cùng thread request.	SRP: chỉ quản lý context; cần clear để tránh rò context giữa request.
shared/error/ErrorCode.java	Enum mã lỗi chung map sang HttpStatus.	Domain/service chỉ cần chọn loại lỗi, RestExceptionHandler quyết định HTTP status theo enum.	OCP hạn chế: thêm loại lỗi bằng cách thêm enum; các consumer không cần đổi nhiều.
shared/error/DomainException.java	RuntimeException cho lỗi nghiệp vụ, giữ ErrorCode và factory notFound/validation/conflict/forbidden/unauthorized.	Service có cách ném lỗi nghiệp vụ thống nhất mà không phụ thuộc trực tiếp vào HTTP.	DIP: domain/application không cần biết ResponseEntity.
shared/error/RestExceptionHandler.java	Global @RestControllerAdvice xử lý DomainException, validation, type mismatch và fallback Exception.	Controller không phải try/catch lặp lại, lỗi REST luôn trả ApiResponse.error cùng format.	SRP tốt ở mức API error translation.

8. Module vehicle

vehicle là module Java hoàn chỉnh nhất hiện tại. Nó đi theo ports/adapters: domain thuần ở giữa, application điều phối use case và định nghĩa port, adapter in là REST, adapter out là JPA/PostgreSQL.

```

vehicle
  domain
  application
    query
    port/in
    port/out
    service
  adapter
    in/rest
    out/persistence

```

Domain

File	Vai trò	Tại sao cần	SOLID / ghi chú
vehicle/domain/Vehicle.java	Record domain biểu diễn xe: id, owner/fleet reference, source, brand/model/version/year/license/seats/transmission/fuel/status/timestamps; có isActive/isHostOwned/isCompanyOwned.	Là model nghiệp vụ sạch, không phụ thuộc JPA annotation hay JSON DTO.	SRP: giữ trạng thái và hành vi nhỏ của Vehicle; DIP: không biết persistence/HTTP.
vehicle/domain/VehicleSource.java	Enum HOST_OWNED, COMPANY_OWNED.	Phân biệt xe host đăng và xe công ty, ảnh hưởng fleet/booking/pricing.	Kiểu hóa giá trị domain thay vì string rời.
vehicle/domain/VehicleStatus.java	Enum DRAFT, PENDING_REVIEW, ACTIVE, INACTIVE, SUSPENDED.	Mô hình hóa lifecycle xe trong admin/marketplace.	Kiểu hóa trạng thái giúp tránh typo.
vehicle/domain/VehicleTransmission.java	Enum MANUAL, AUTOMATIC.	Dữ liệu kỹ thuật xe để filter/display.	Kiểu hóa option domain.
vehicle/domain/VehicleFuelType.java	Enum GASOLINE, DIESEL, ELECTRIC, HYBRID.	Dữ liệu kỹ thuật xe để filter/display.	Kiểu hóa option domain.

Application layer

File	Vai trò	Tại sao cần	SOLID / ghi chú
vehicle/application/query/Enums.java	Utility package-private parse string sang enum, invalid thì trả null.	Tập trung logic parse query param source/status cho ListVehiclesQuery/PageVehiclesQuery.	SRP tốt nhưng hành vi invalid -> null có thể làm API quá im lặng; nếu strict hơn nên ném DomainException.validation.
vehicle/application/query/ListVehiclesQuery.java	Query object cho list không phân trang, parse source/status sang enum.	Gom input use case thành object rõ nghĩa thay vì truyền tham số rời.	SRP: chuẩn hóa input cho một use case.
vehicle/application/query/PageVehiclesQuery.java	Query object cho list phân trang: q/source/status/page/size, normalize q, page 1-based, size mặc định 20 và max 100.	Chặn input paging xấu trước khi xuống persistence; giữ rule pagination ở application query.	SRP tốt; OCP nếu thêm filter mới thì mở rộng query object và adapter tương ứng.
vehicle/application/port/in/ListVehiclesUseCase.java	Inbound port @FunctionalInterface: List<Vehicle> handle(ListVehiclesQuery).	Controller phụ thuộc contract use case thay vì class service cụ thể.	DIP/ISP: interface nhỏ, đúng một use case.

File	Vai trò	Tại sao cần	SOLID / ghi chú
vehicle/application/port/in/PageVehiclesUseCase.java	Inbound port @FunctionalInterface: PageResponse<Vehicle> handle(PageVehiclesQuery).	Tách use case phân trang khỏi use case list thường.	ISP: client chỉ phụ thuộc method cần dùng.
vehicle/application/port/out/LoadVehiclePort.java	Outbound port để application load vehicle list/page từ storage.	VehicleQueryService không cần biết JPA/PostgreSQL/caching; chỉ biết cần dữ liệu Vehicle.	DIP rõ nhất trong module vehicle: high-level service phụ thuộc abstraction.
vehicle/application/service/VehicleQueryService.java	Service @Service, @Transactional(readOnly=true), implement ListVehiclesUseCase và PageVehiclesUseCase, delegate sang LoadVehiclePort.	Điểm điều phối use case đọc vehicle; transaction read-only đặt ở application layer.	SRP: orchestration; DIP: phụ thuộc LoadVehiclePort; hiện logic còn mỏng nhưng đúng vị trí để thêm rule đọc.

REST adapter

File	Vai trò	Tại sao cần	SOLID / ghi chú
vehicle/adapter/in/rest/AdminVehicleController.java	REST controller /api/v1/admin/vehicles và /paged; nhận query params, tạo query object, gọi use case, map response, wrap ApiResponse.	Là adapter inbound HTTP cho admin web.	SRP: HTTP translation; DIP: phụ thuộc inbound ports, không gọi repository trực tiếp.
vehicle/adapter/in/rest/VehicleApiMapper.java	Mapper package-private từ domain Vehicle sang VehicleResponse; enum thành name string.	Giữ API DTO tách khỏi domain model.	SRP: mapping API; tránh domain phụ thuộc format JSON.
vehicle/adapter/in/rest/VehicleResponse.java	Record DTO trả về frontend với các field xe và enum dạng String.	Định nghĩa contract JSON ổn định cho admin UI.	SRP: chỉ là API output shape.

Persistence adapter

File	Vai trò	Tại sao cần	SOLID / ghi chú
vehicle/adapter/out/persistence/VehicleJpaEntity.java	JPA entity map bảng vehicle.vehicles, schema vehicle; dùng @Entity/@Table/@Column/@Enumerated và Lombok getter/setter/no-args constructor protected.	JPA cần entity mutable để hydrate row DB; tách khỏi domain record immutable.	SRP: persistence mapping. Domain không bị annotation JPA chi phối.
vehicle/adapter/out/persistence/VehicleJpaRepository.java	Spring Data repository extends JpaRepository và JpaSpecificationExecutor; có native query search với ILIKE, source/status filter, order by created_at desc và countQuery.	Cung cấp CRUD/specification và query search tối ưu cho PostgreSQL trigram index.	SRP repository; nhưng đây là infrastructure nên được giấu sau LoadVehiclePort.
vehicle/adapter/out/persistence/VehicleJpaSpecifications.java	Tạo Specification từ ListVehiclesQuery: thêm predicate source/status khi có filter.	Tránh viết nhiều method findBySourceAndStatus; filter động gọn hơn.	OCP: thêm filter bằng thêm predicate, không nổ số lượng method repository.

File	Vai trò	Tại sao cần	SOLID / ghi chú
vehicle/adapters/out/persistence/VehiclePersistenceAdapter.java	@Component implement LoadVehiclePort; loadVehicles dùng specification + sort createdAt DESC; loadVehiclePage gọi repository.search rồi convert Page sang PageResponse.	Nối application port với JPA infrastructure.	DIP: adapter phụ thuộc framework, application chỉ thấy port.
vehicle/adapters/out/persistence/VehiclePersistenceMapper.java	Mapper từ VehicleJpaEntity sang domain Vehicle.	Không trả entity persistence lên domain/application/API; giảm coupling.	SRP mapping persistence-domain.

9. Test và thư mục khung

File / thư mục	Vai trò	Tại sao cần	SOLID / ghi chú
src/test/java/com/ares/car_rental_monolith/CarRentalMonolithApplicationTests.java	@SpringBootTest contextLoads() mặc định.	Smoke test kiểm tra Spring context có khởi động được không.	Chưa test SOLID trực tiếp; nên bổ sung unit/integration test cho VehicleQueryService, repository search và controller contract.
src/test/java/.../common và modules/*	Các thư mục test module đang có trong workspace nhưng chưa có file test.	Dành chỗ cho test theo từng module nghiệp vụ.	Khi mở rộng, nên test theo boundary module để giữ SRP/DIP.

Các module Java đang là khung rỗng

Các thư mục module chưa có file Java vẫn có ý nghĩa định hướng kiến trúc. Chúng cho biết team muốn phát triển tiếp theo module nghiệp vụ thay vì gom tất cả controller/service/repository vào một package chung.

Thư mục	Vai trò dự kiến	Tại sao cần
modules/admin/{api,application,domain,infrastructure}	Khung cho admin/backoffice Java, hiện chưa có class.	Tương ứng schema admin: audit logs, tasks, dashboard snapshots.
modules/booking/{api,application,domain,infrastructure}	Khung cho booking Java, hiện chưa có class.	Sẽ chứa lifecycle đặt xe, status history, pickup/return checklist.
modules/customer/{api,application,domain,infrastructure}	Khung cho customer Java, hiện chưa có class.	Sẽ chứa profile renter/host, KYC, address.
modules/driver/{api,application,domain,infrastructure}	Khung cho driver Java, hiện chưa có class.	Sẽ chứa hồ sơ tài xế, lịch rảnh, assignment.
modules/fleet/{api,application,domain,infrastructure}	Khung cho fleet Java, hiện chưa có class.	Sẽ chứa xe công ty, branch, maintenance, insurance, inspection.
modules/identity/{api,application,domain,infrastructure}	Khung cho identity Java, hiện chưa có class.	Sẽ chứa user, role, permission, login session.
modules/notification/{api,application,domain,infrastructure}	Khung cho notification Java, hiện chưa có class.	Sẽ chứa template, notification, delivery log, device.
modules/payment/{api,application,domain,infrastructure}	Khung cho payment Java, hiện chưa có class.	Sẽ chứa payment intent, transaction, refund, payout.
modules/pricing/{api,application,domain,infrastructure}	Khung cho pricing Java, hiện chưa có class.	Sẽ chứa price plan, promotion, quote, quote item.

Thư mục	Vai trò dự kiến	Tại sao cần
modules/review/{api,application,domain,infrastructure}	Khung cho review Java, hiện chưa có class.	Sẽ chứa review, reply, report.
infra/k8s	Thư mục hạ tầng Kubernetes đang rỗng.	Dành cho manifest/deployment/service/ingress nếu deploy lên Kubernetes.
docs/api	Thư mục tài liệu API đang rỗng.	Dành cho OpenAPI, Postman collection hoặc mô tả endpoint.

10. Tài liệu kiến trúc trong docs

File	Vai trò	Tại sao cần	SOLID / ghi chú
docs/backend-source-explanation.md	Tài liệu Markdown giải thích backend đã có sẵn trong repo.	Là bản ghi chú dạng text; tài liệu PDF này là bản cập nhật có thêm SOLID và đóng gói dễ đọc hơn.	Tài liệu hỗ trợ hiểu thiết kế, không ảnh hưởng runtime.
docs/architecture/database.md	Mô tả nguyên tắc database: PostgreSQL, schema-per-module, không FK vật lý xuyên business schema, dùng UUID reference và snapshot.	Giúp team giữ ranh giới module khi viết migration/code mới.	Ủng hộ SRP/DIP ở cấp kiến trúc: module sở hữu schema riêng và giảm coupling vật lý.
docs/architecture/database-diagram.md	ERD dạng Mermaid cho toàn bộ schema.	Giúp đọc quan hệ bảng nhanh hơn so với chỉ đọc SQL migration.	Tài liệu hóa boundary giữa modules.
docs/architecture/database-diagram.doc	Bản Word cũ hoặc compatibility của database diagram.	Dễ chia sẻ với người dùng Word hoặc stakeholder không đọc Markdown.	Không tham gia runtime.
docs/architecture/database-diagram.docx	Bản Word hiện đại của database diagram.	Phục vụ bàn giao/tài liệu hóa kiến trúc DB.	Không tham gia runtime.
docs/architecture/car_rental_architecture.png	Hình minh họa kiến trúc tổng quan hệ thống thuê xe.	Hỗ trợ onboarding nhanh bằng trực quan.	Không tham gia runtime.
docs/architecture/database-booking-flow.png	Hình minh họa flow database liên quan booking.	Giúp hiểu booking đi qua customer/vehicle/pricing/payment như thế nào.	Không tham gia runtime.
docs/architecture/database-overview.png	Hình tổng quan database.	Phục vụ đọc schema theo nhóm nghiệp vụ.	Không tham gia runtime.
docs/architecture/database-table-map.png	Bản đồ table theo module.	Giúp tra cứu table nhanh khi mở migration.	Không tham gia runtime.

database.md và database-diagram.md là hai file quan trọng nhất trong nhóm docs vì chúng ghi rõ quy tắc không tạo foreign key vật lý xuyên business schema. Quy tắc này ảnh hưởng trực tiếp đến cách viết code application: code phải kiểm soát consistency bằng use case và transaction thay vì dựa hoàn toàn vào constraint DB xuyên module.

11. SOLID trong backend hiện tại

S - Single Responsibility Principle

Nguyên tắc: một class nên có một lý do chính để thay đổi. Code hiện tại áp dụng khá rõ trong module vehicle. AdminVehicleController thay đổi khi HTTP contract thay đổi. VehicleQueryService thay đổi khi use case đọc xe

thay đổi. VehiclePersistenceAdapter thay đổi khi cách lấy dữ liệu từ storage thay đổi. VehicleApiMapper và VehiclePersistenceMapper thay đổi khi mapping API/persistence thay đổi.

Ví dụ	Đánh giá
AdminVehicleController	Chỉ nhận request, tạo query, gọi use case, trả response. Không chứa SQL.
VehicleJpaRepository	Chỉ biết cách query DB. Không biết HTTP response.
RestExceptionHandler	Chỉ chuyển exception thành response lỗi REST.
P6SpySqlFormatter	Chỉ format SQL log; không can thiệp query.

O - Open/Closed Principle

Nguyên tắc: code nên mở để mở rộng nhưng hạn chế sửa code cũ. Hiện tại nếu muốn thêm persistence khác, có thể tạo adapter mới implement LoadVehiclePort. Nếu muốn thêm filter cho list vehicles, có thể mở rộng query object và VehicleJpaSpecifications. Nếu muốn đổi response DTO, sửa mapper/DTO mà không làm domain phụ thuộc API.

Điểm cần lưu ý: native query search trong VehicleJpaRepository phải sửa nếu thêm filter cho endpoint /paged. Đây là bình thường với SQL cụ thể, nhưng nên kiểm soát bằng test repository để tránh regression.

L - Liskov Substitution Principle

Nguyên tắc: implementation của interface phải thay thế được abstraction mà không làm hỏng kỳ vọng của client. ListVehiclesUseCase, PageVehiclesUseCase và LoadVehiclePort đang là các contract nhỏ, dễ thay thế bằng implementation fake khi test. Nếu sau này tạo CachedVehiclePersistenceAdapter hay JdbcVehiclePersistenceAdapter, chúng phải giữ cùng semantics: filter source/status giống nhau, page 1-based ở response, sort mặc định hợp lý và không trả null list.

I - Interface Segregation Principle

Nguyên tắc: client không nên phụ thuộc interface chứa method không dùng. Code hiện tại tách ListVehiclesUseCase và PageVehiclesUseCase thành hai interface riêng. Đây là điểm tốt vì controller có thể inject đúng capability cần dùng, test cũng dễ fake từng use case. LoadVehiclePort hiện có hai method đọc vehicle; vẫn ổn vì cùng nhóm read model, nhưng nếu sau này thêm create/update/delete thì nên tách thành SaveVehiclePort, DeleteVehiclePort hoặc command ports riêng.

D - Dependency Inversion Principle

Nguyên tắc: module cấp cao phụ thuộc abstraction, không phụ thuộc detail cấp thấp. Đây là nguyên tắc thể hiện rõ nhất trong code vehicle. Controller phụ thuộc inbound use case interfaces. VehicleQueryService phụ thuộc LoadVehiclePort. JPA nằm ở adapter out và implement port. Nhờ vậy application layer không import JpaRepository, EntityManager, @Query hoặc ResponseEntity.

```
Controller -> ListVehiclesUseCase / PageVehiclesUseCase
VehicleQueryService -> LoadVehiclePort
VehiclePersistenceAdapter -> VehicleJpaRepository
Domain Vehicle -> không phụ thuộc Spring/JPA/HTTP
```

Các điểm nên cải thiện để SOLID mạnh hơn

- Enums.parse hiện invalid value thành null, nghĩa là API bỏ filter sai thay vì báo INVALID_PARAMETER. Với admin API, hành vi strict thường tốt hơn.
- Module vehicle dùng package adapter/in/out, còn các module khung khác dùng api/application/domain/infrastructure. Nên thống nhất naming style để giảm cognitive load.
- VehicleQueryService hiện chỉ delegate, nhưng vị trí này đúng để thêm rule nghiệp vụ đọc. Nếu mãi chỉ delegate, có thể xem xét đơn giản hóa nhưng không nên để controller gọi repository.
- Chưa có test cho repository search, mapper, controller response và pagination boundary. SOLID chỉ có giá trị khi có test bảo vệ contract.
- Nếu LoadVehiclePort mở rộng sang nhiều trách nhiệm ghi/xóa/tìm kiếm phức tạp, nên tách interface nhỏ hơn theo ISP.

12. Luồng API vehicle hiện tại

GET /api/v1/admin/vehicles

Endpoint list không phân trang nhận source và status optional. Controller gọi ListVehiclesQuery.from để parse string thành enum, sau đó gọi ListVehiclesUseCase. Persistence dùng Specification để thêm predicate nếu filter khác null, sort createdAt DESC, map entity sang domain, rồi controller map domain sang VehicleResponse và wrap bằng ApiResponse.success với code VEHICLE_LIST_FETCHED.

GET /api/v1/admin/vehicles/paged

Endpoint phân trang nhận q, source, status, page, size. PageVehiclesQuery chuẩn hóa page tối thiểu 1, size mặc định 20 và tối đa 100, q trim/lowercase. Repository dùng native SQL ILIKE để search brand/model/version/license_plate, kết hợp source/status và countQuery. V031 tạo trigram index để query này mở rộng tốt hơn khi dữ liệu tăng.

Input	Rule hiện tại
page	Nếu null hoặc < 1 thì thành 1. Response page là 1-based.
size	Nếu null hoặc < 1 thì thành 20; nếu quá lớn thì max 100.
q	Null/blank thành chuỗi rỗng, nghĩa là không search.
source/status	Parse enum case-insensitive; invalid hiện thành null, nghĩa là bỏ filter.

13. Kết luận kỹ thuật

Backend hiện có nền tảng tốt cho modular monolith: schema-per-module, Flyway kiểm soát DB, shared response/error handling, CORS cho admin web, SQL logging có context, và module vehicle đã tách domain/application/adapter rõ ràng. Phần còn thiếu lớn nhất không phải là cấu trúc mà là độ phủ implementation và test: đa số module nghiệp vụ ngoài vehicle mới có database schema, chưa có Java code.

Khi phát triển tiếp, nên giữ nguyên hướng ports/adapters cho từng module, thống nhất naming giữa vehicle và các module còn lại, thêm validation strict cho query param, bổ sung test theo từng boundary, và tiếp tục dùng migration mới thay vì sửa migration cũ đã chạy.